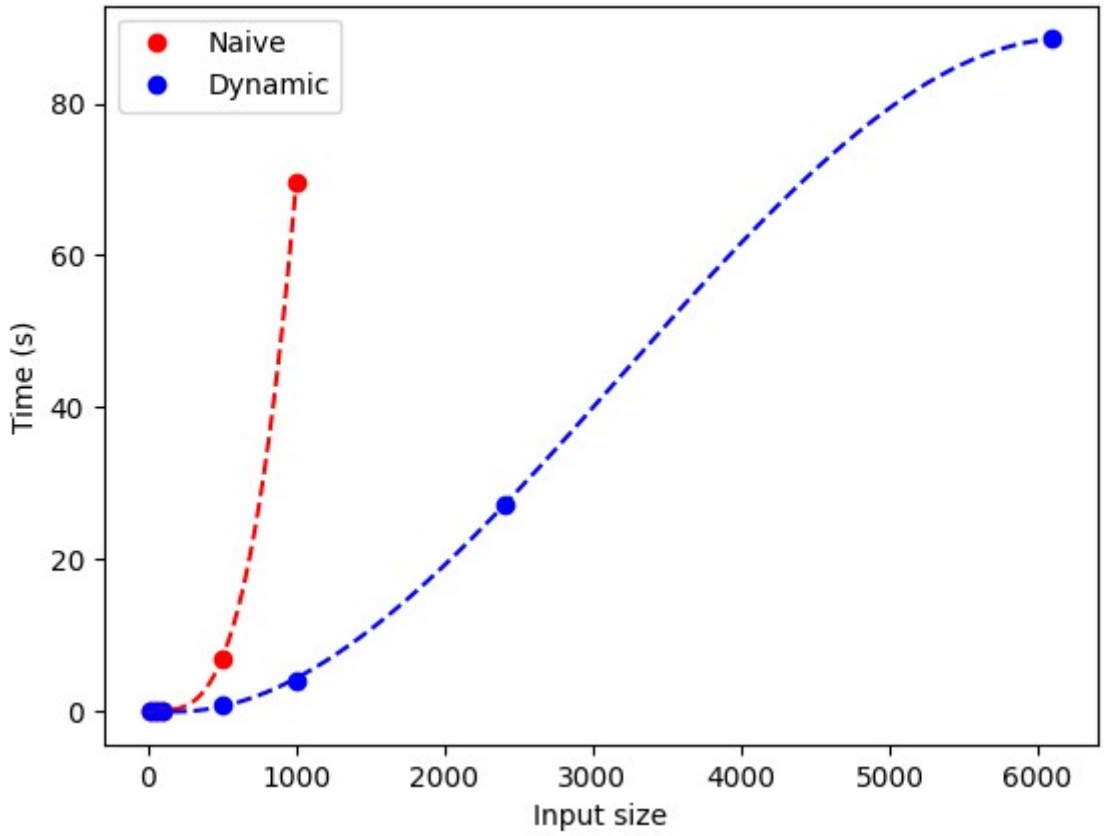
Benchmarking

Once I had the program working, I ran the remove seam operation on 5 consecutive seams with timers directly before and after the calling of the best seam function. I took the average of the 5 runs and plotted against the various images I tested on. I used the pixilart.com website that was suggested to create images that were 10, 50, 100, 500, and 1000 square pixels. In order to really push the dynamic method, I also found a few really large images online, one of clouds which was ~3200x2000, and one of Perth which was ~7500x5000. The naive recursive implementation led to the first set of averages, and the dynamic implementation the second:

10x10: 0.001381 seconds
50x50: 0.021078 seconds
100x100: 0.087698 seconds
500x500: 6.935343 seconds
1000x1000: 69.569059 seconds

10x10: 0.000497 seconds
50x50: 0.009713 seconds
100x100: 0.038419 seconds
500x500: 0.926438 seconds
1000x1000: 4.050124 seconds
clouds: 27.101862 seconds
butte: 88.469311 seconds



Analysis

First for the recursive implementation, the asymptotic complexity could be O(width * 3^{height}), since for each pixel in a row, the function will explore up to three recursive paths it can travel. Without any memoization, recursion will take all paths, so the above complexity remains, but with memoization the actual complexity is down to O(width * height), since the number of subproblems is now bounded by the number of pixels. No subproblem will be repeated due to the memoization, but the function does still call itself in order to check for memos. The work done at each step is constant, comparing at most three values (O(1)). Together this means $T(pixel_{i, j}) = 3 * (T(pixel_{i, j+1}) + O(1))$.

The dynamic implementation's asymptotic complexity is actually the same, O(width * height), since each pixel still needs to be checked at some point. The difference in performance can be seen in the recurrence relations. The dynamic version's relation is $T(pixel_{i, j}) = T(width, height - 1) + O(width)$.

Despite the complexity of each solution being the same, these results still make sense because there are some factors that recursion deals with but dynamic programming doesn't have to. The practical considerations of calling functions and then returning from them (3 times for every pixel), more optimal cache usage, and having to check for memos makes the recursion take much longer than an approach that just has to compute each row iteratively, likely has related data closer together in cache, and doesn't have to call itself repeatedly. In particular the calling of 3 recursive functions for each pixel is an enormous overhead that far outweighs the larger constant component of the dynamic recurrence relation in the long run.